Name :- Sara Munir Alam
Roll# :- 24K-0573
Class :- BCS-5E Date:

# DAA [Assignment 2]

## Q1) Assumption

- P(n vertices) and Q (m vertices) are simple polygons.
- They are closed regions, so touching count as intersecting. For "touching doesn't" count.
- Three possible outcome PROPER (segment cross each other), TOUCH (they touch at an endpoint or overlap along a line), NONE (They don't intersect).
- cross product works like this

$$\text{cross}(A,B,C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x)$$

Result: +ve : point lies on one side of the directed line.
-ve : point lies on other side
zero : point on same line

Algorithm.

```
SEG-RELATION (p1, p2, p3, p4):  // classify segments p1p2 and p3p4
  d1 = cross(p3, p4, p1);   d2 = cross(p3, p4, p2)
  d3 = cross(p1, p2, p3);   d4 = cross(p1, p2, p4)
  if sign(d1) * sign(d2) < 0 and sign(d3) * sign(d4) < 0 : return PROPER
  if d1 == 0 and ON-SEG(p3, p4, p1): return TOUCH
  if d2 == 0 and ON-SEG(p3, p4, p2): return TOUCH
  if d3 == 0 and ON-SEG(p1, p2, p3): return TOUCH
  if d4 == 0 and ON-SEG(p1, p2, p4): return TOUCH
  return NONE

ON-SEG (a, b, p):   // p is already collinear with a, b
  return min(a.x, b.x) <= p.x <= max(a.x, b.x) and min(a.y, b.y) <=
         p.y <= max(a.y, b.y)

POLY-INTERSECT (P[0..n-1], Q[0..m-1]):
  1 if BBOX(P) and BBOX(Q) are disjoint : return FALSE // quick reject
```

Date:

```
2  for i=0 to n-1:
3      for j=0 to m-1:
4          if SEG-RELATION(P[i], P[(i+1)%n], Q[j], Q[(j+1)%m]) !=
4          NONE: return TRUE
5  if: POINT-IN-POLY(P[0], Q) != OUTSIDE: return TRUE   // P inside Q
6  if POINT-IN-POLY(Q[0], P) != OUTSIDE: return TRUE    // Q inside P
7  return FALSE
```

| Cases Handled | caught by |
|---|---|
| • Edges cross properly | PROPER line 4 |
| • Vertex of one lies on an edge of the other | TOUCH, line 4 |
| • shared vertex or shared edge | TOUCH, line 4 |
| • collinear overlapping edges (all $d=0$) | the ON-SEG tests, line 4 |
| • P entirely inside Q, or Q inside P | line 5 – 6 |
| • Identical polygons | line 4 |
| • Disjoint | falls through to line 7 |

Justification

• Boundary intersection: if the boundaries of (P) and (Q) intersect, at least one edge of (P) intersects an edge of (Q). SEG-RELATION correctly detects proper crossing, touching, and collinear overlaps.

• Containment: if the boundaries are disjoint, the Jordan curve Theorem implies that the entire boundary of one polygon lies either inside or outside the other polygon. Hence testing one vertex of each polygon is sufficient to detect it.

• Disjoint polygons: If no pair of edges intersects and both tested vertices lies outside the other polygon, neither polygons contains the other. Since partial overlap without boundary intersection is impossible for simple polygons, the polygons are disjoint, and the algorithm correctly returns false.

Date:

(Q2) Given

- Vertices $v_0 \dots v_{n-1}$ in CCW order.
- A kernel point $q$, meaning every vertex is visible from $q$.

Key fact: Every ray from $q$ hits the boundary exactly once, so the vertices in polygon order are already sorted by polar angle around $q$. The sort, which is the $O(n \log n)$ bottleneck of Graham's scan, is therefore free.

Algorithm

STAR-HULL (V[0..n-1] CCW order around kernel point q):

```
1   s = index of vertex with min y (ties: min x)
2   rotate V so that V[0] = that vertex
3   S = [V[0], V[1]]
4   for i = 2 to n-1:
5       while |S| >= 2 and cross(S[top-1], S[top], V[i]) <= 0:
6           pop S
7       push V[i]
8   while |S| >= 3 and cross(S[top-1], S[top], V[0]) <= 0: pop S
9   return S
```

V[0] is the lowest vertex, so it is an extreme point and is never popped. The sequence is angularly sorted around the interior point $q$.

The scan invariant is that the stack is always a strictly convex chain of left turns. Each right or collinear turn pops a vertex that lies inside or on the hull of the processed prefix. Using <=0 removes collinear, so the output is the minimum hull.

Standard Algorithm needs modification: Standard Graham scan, does sorting by angle around the lowest vertex costs $O(n \log n)$, which is unnecessary because the vertices are already angularly ordered around. Vertex order, meaning Polygon order is not generally sorted around the lowest vertex. Use $q$ as

Date:

as the angular reference, not the pivot.
The scan is only correct when the points are in angular order around a point inside the hull (the kernel point) & the start vertex is a true hull vertex.

## Question 3

Method 1 : Ray casting (even-odd rule) : Shoot a ray from p to +x and count edge crossings. An odd count means inside.

```
RAY-CAST (p, V[0..n-1]):
    c = 0
    for i = 0 to n-1:
        a = V[i];  b = V[(i+1) % n]
        if ON-SEG (a, b, p) and cross (a, b, p) == 0: return BOUNDARY
        if (a.y > p.y) != (b.y > p.y):   // half-open rule
            xint = a.x + (p.y - a.y) * (b.x - a.x) / (b.y - a.y)
            if xint > p.x :
                c = c + 1
    return (c odd) ? INSIDE : OUTSIDE
```

The half-open test $(a.y > p.y) \neq (b.y > p.y)$ handles the degeneracy where the ray passes exactly through a vertex. The vertex is counted once for an edge that goes through it and not at all for an edge that only touches it.

Method 2 : Winding Number.

```
Winding (p, V):
    w = 0
    for each edge a → b :
```

Date:

if ON-SEG (a, b, p) and cross (a, b, p) == 0: return BOUNDARY
if a.y <= p.y:
    if b.y > p.y and cross (a, b, p) > 0:
        w++ // upward edge, p on left
else:
    if b.y <= p.y and cross (a, b, p) < 0:
        w-- // downward edge, p on right
return (w != 0) ? INSIDE : OUTSIDE

for a simple polygon, w = ±1 inside and 0 outside. It also works for self intersecting polygons.

Question 4

CLUSTER-HULLS (Pts[1..n], ε):
Build graph G:
    connect points at distance <= ε
components C1.. Cm via DFS/union find
for each Ck:
    if |Ck| == 1:
        Hk = Point (p)
    else if all of Ck collinear:
        Hk = SEGMENT (two extreme points)
    else:
        Hk = GRAHAM-SCAN (Ck)
for each pair (i<j):
    if HULL-INTERSECT (Hi, Hj) Hi∩n mark (i, j) OVERLAPPING
if no pair is marked: report m disjoint convex polygons H1.. Hm
else: report the overlapping group; do NOT merge silently.

Date:

HULL-INTERSECT (A, B) :

POINT vs POINT : return A == B

POINT vs SEGMENT : return ON-SEG (B.p, B.q, A) and cross (B.p, B.q, A) == 0

POINT vs POLYGON : return POINT-IN-POLY (A, B) != OUTSIDE

SEGMENT vs SEGMENT : return SEG-RELATION (A, B) != None

SEGMENT vs POLYGON : return any edge of B meets A OR POINT-IN-POLY (A.p, B) != OUTSIDE

POLYGON vs POLYGON : return POLY-Intersect (A, B)

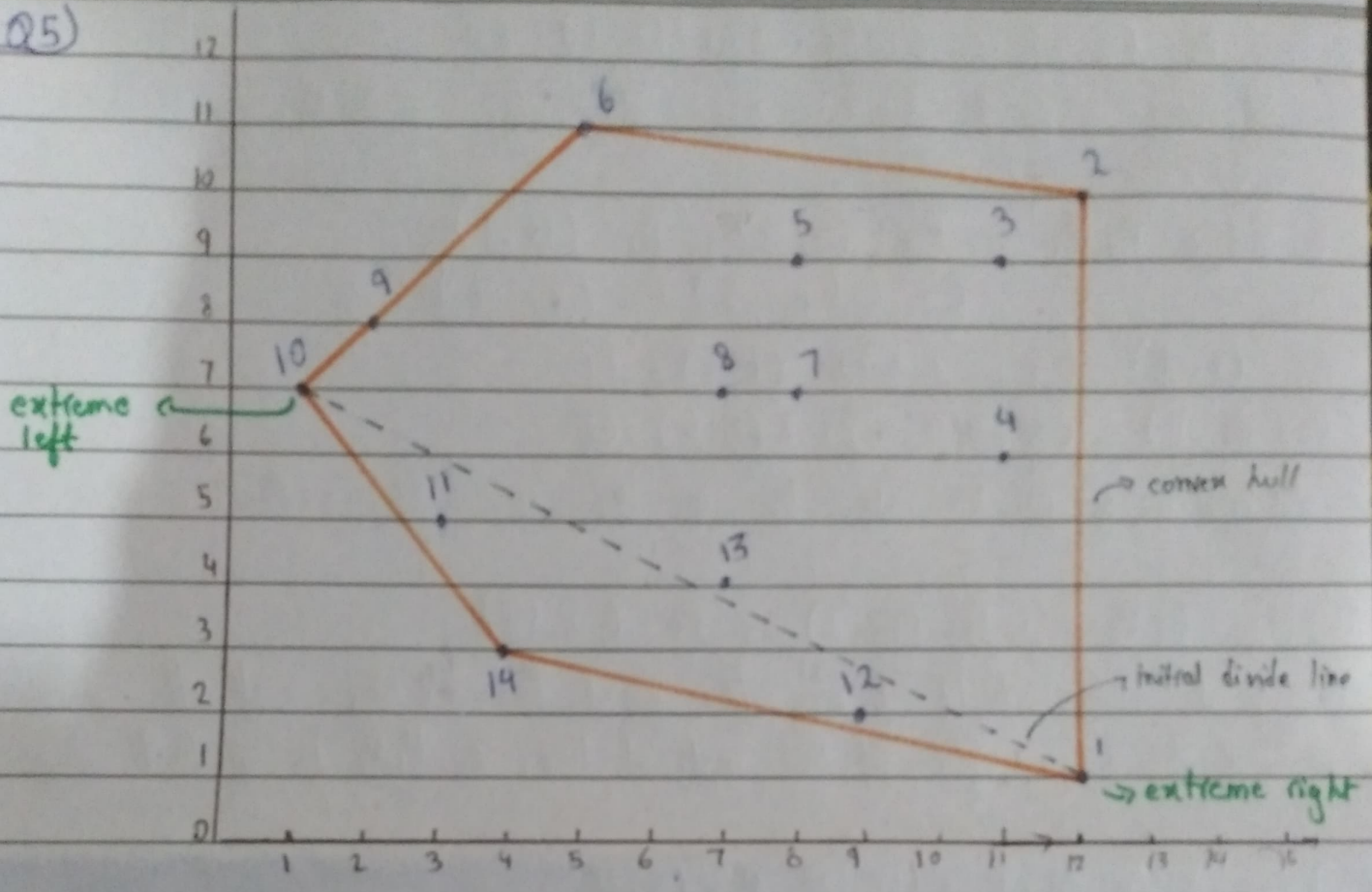
explanation :

The case table covers every pairing of point, segment and polygon, so no call ever treats a point or segment as polygon.

No silent merging. A cluster is only a connected component at scale $\varepsilon$, so the results depends on $\varepsilon$. If hulls overlap, the correct report is that this $\varepsilon$ does not yield disjoint convex groups. The user can then change $\varepsilon$ or the clustering method.

Date:

Q5)




As per sir statement, I have assumed points co-ordinates.

Method 1:- Brute Force (Time complexity: $O(n^3)$ ← as there are $O(n^2)$ pairs of points, and each may require checking with other point $O(n)$)

Principle:- For every pair of points $P_i, P_j$ check whether all points lie on the same side of the line joining them. If yes the pair forms a convex hull edge.

$$C = (x_j - x_i)(y_k - y_i) - (y_j - y_i)(x_k - x_i)$$

$C > 0$: $P_k$ lies to left of directed line

$C < 0$: $P_k$ lies to the right

$C = 0$: The points are collinear.

Test hull edges.

Edge: $P_{10} P_{14}$ $P_{10} = (1,7)$, $P_{14} = (4,3)$

$$C = 3(y_k - 7) + 4(x_k - 1)$$

For $P_1 = (12,1)$ → $C = 3(1-7) + 4(12-1) = 26$

Date:

For $P_6 = (5, 11)$ : $C = 3(11-7) + 4(5-1) = 28$

All remaining points also have non negative cross products so $P_{10}P_{14}$ is hull edge.

Edge $P_{14}P_1 \rightarrow P_{14} = (4, 3)$, $P_1 = (12, 1)$

$C = 8(y_k - 3) + 2(x_k - 4)$

$P_{10} = (1, 7)$ $C = 8(7-3) + 2(1-4) = 26$

$P_6 = (5, 11)$ $C = 8(11-3) + 2(5-4) = 66$

All remaining points also lie on the same sid. $\therefore P_{14}P_1$ is a hull edge

Edge $P_1P_2 \rightarrow P_1 = (12, 1)$, $P_2 = (12, 10)$

Both points have $x = 12$. Every other point has $x \leq 12$, so all other points lie on the same side of this vertical line. $\therefore P_1P_2$ is a hull edge.

Edge $P_2P_6 \rightarrow P_2 = (12, 10)$, $P_6 = (5, 11)$

$C = -7(y_k - 10) - (x_k - 12)$

$P_1 = (12, 1)$ $C = -7(1-10) - (12-12) = 63$

$P_{10} = (1, 7)$ $C = -7(7-10) - (1-12) = 32$

All remaining points lie on the same side. $\therefore P_2P_6$ is a hull edge

Edge $P_6P_{10} \rightarrow P_6 = (5, 11)$, $P_{10} = (1, 7)$

$C = -4(y_k - 11) + 4(x_k - 5)$

$P_1 = (12, 1)$ $C = -4(1-11) + 4(12-5) = 68$

$P_9 = (2, 8)$ $C = -4(8-11) + 4(2-5) = 0$

Thus $P_9$ is collinear with $P_6$ and $P_{10}$. Remaining point lie on the same side of this line, so $P_6P_{10}$ is a hull edge.

Final Result :- ~~$P_{10}$-$P_1$~~ $P_{10}$-$P_{14}$, $P_{14}$-$P_1$, $P_1$-$P_2$, $P_2$-$P_6$, $P_6$-$P_{10}$

$\therefore P_{10} \rightarrow P_{14} \rightarrow P_1 \rightarrow P_2 \rightarrow P_6 \rightarrow P_{10}$

Date:

# Method 2 :- Quick hull

Leftmost Extreme Point = $P_{10} = (1,7)$

Rightmost Extreme Point = $P_1 = (12,1)$ { was tie with $P_2$ for both x was x-coordinate 12, so I choose $P_1$

The line LR divides the points into two grps.

Use cross product : $C(L,R,P_i) = (12-1)(y_i-7) - (1-7)(x_i-1)$

$C(L,R,P_i) = 11(y_i-7) + 6(x_i-1)$

+ve value indicate one side of the line, and -ve values indicates the other.

- Find the farthest point from the initial line.

| Point | Cross Product | Side | Point | Cross Product | Side |
|---|---|---|---|---|---|
| $P_2(12,1)$ | 99 | +ve | $P_8(7,7)$ | 36 | +ve |
| $P_3(11,9)$ | 82 | +ve | $P_9(2,8)$ | 17 | +ve |
| $P_4(11,6)$ | 49 | +ve | $P_{10}(3,5)$ | -10 | -ve |
| $P_5(8,9)$ | 64 | +ve | $P_{12}(9,2)$ | -7 | -ve |
| $P_6(5,11)$ | 68 | +ve | $P_{13}(7,4)$ | 3 | +ve |
| $P_7(8,7)$ | 42 | +ve | $P_{14}(4,3)$ | -26 | -ve |

largest val at +ve side is Point 2 (12,10). { first candidat point selected by Quick Hull

larget val at -ve side is Point 14 (4,3).

Recursive process the upper side

for line $P_{10}P_2$ where $P_{10} = (1,7)$, $P_2 = (12,12)$

$C(P_{10},P_2,P_x) = 11(y_i-7) - 3(x_i-1)$

| Point | Cross Product | Side |
|---|---|---|
| $P_5(8,9)$ | 1 | +ve |
| $P_6(5,11)$ | 32 | +ve |
| $P_9(2,8)$ | 8 | +ve |

Selected Point ⇒ $P_6 = (5,11$

Current upper chain

$P_{10} \rightarrow P_6 \rightarrow P_2 \rightarrow P_1$

Date:

No addition points lie outside the resulting upper boundary.

Recursively Process lower side

Line $P_1P_4$ : No points lie outside this edge

Line $P_{14}P_{10}$ :- No points lie outside this edge.

∴ Lower chain is $P_1 \rightarrow P_{14} - P_{10}$

Combine Chain: $P_{10} \rightarrow P_{14} \rightarrow P_1 \rightarrow P_2 \rightarrow P_6 \rightarrow P_{10}$ } convex hull

Time complexity:

Avg case :- $O(n \log n)$ Worst Case = $O(n^2)$

## Method 3 :- Divide & Conquer.

→ Sort points by increasing x-coordinate (in case of tie sort by increasing y-coordinate).

sequence ⇒ $P_{10}, P_9, P_{11}, P_{14}, P_6, P_{13}, P_8, P_7, P_5, P_{12}, P_4, P_3, P_1, P_2$

→ Divide into two subsets.

Left Subset ⇒ $\{P_{10}, P_9, P_{11}, P_{14}, P_6, P_{13}, P_8\}$

Right Subset ⇒ $\{P_7, P_5, P_{12}, P_4, P_3, P_1, P_2\}$

For left subset lower hull

| Point | A | B | Cross | action (B) |
|---|---|---|---|---|
| $P_{11}$ | $P_{10}$ | $P_9$ | -4 | Remove |
| $P_4$ | $P_{10}$ | $P_{11}$ | -2 | Remove |
| $P_6$ | $P_{10}$ | $P_{14}$ | 28 | keep |
| $P_{13}$ | $P_{14}$ | $P_6$ | -23 | Remove |
| $P_{13}$ | $P_{10}$ | $P_{14}$ | 15 | keep |
| $P_8$ | $P_{14}$ | $P_{13}$ | 9 | keep |

For left subset upper hull

| Point | A | B | Cross | Decision (B) |
|---|---|---|---|---|
| $P_6$ | $P_8$ | $P_{13}$ | -6 | Remove |
| $P_{14}$ | $P_8$ | $P_6$ | -20 | keep |
| $P_{11}$ | $P_6$ | $P_{14}$ | -10 | Remove |
| $P_{11}$ | $P_8$ | $P_6$ | 20 | Keep |
| $P_9$ | $P_6$ | $P_{11}$ | -12 | Remove |
| $P_9$ | $P_8$ | $P_6$ | 18 | keep |
| | | $P_{10}$ | 0 | Remove |
| | | $P_8$ | 24 | keep |

Date:

~~Upper chain: $P_{10} \to P_{14} \to P_{13} \to P_8$~~

Combined chain :- $H_L = P_{10} \to P_{14} \to P_{13} \to P_8 \to P_6 \to P_{10}$

Find Right subset lower hull

| Point | A | B | Cross | Decision (B) |
|---|---|---|---|---|
| $P_{12}$ | $P_7$ | $P_5$ | -2 | Remove |
| $P_4$ | $P_7$ | $P_{12}$ | 14 | Keep |
| $P_3$ | $P_{12}$ | $P_4$ | 6 | Keep |
| $P_1$ | $P_4$ | $P_3$ | -3 | Remove |
| $P_1$ | $P_{12}$ | $P_4$ | -14 | Remove |
| $P_1$ | $P_7$ | $P_{12}$ | 14 | Keep |
| $P_2$ | $P_{12}$ | $P_1$ | 27 | Keep |

Find Right subset upper hull

| Point | A | B | Cross | Decision (B) |
|---|---|---|---|---|
| $P_3$ | $P_2$ | $P_1$ | -9 | Remove |
| $P_4$ | $P_2$ | $P_3$ | 3 | Keep |
| $P_{12}$ | $P_3$ | $P_4$ | -6 | Remove |
| $P_{12}$ | $P_2$ | $P_3$ | 5 | Keep |
| $P_5$ | $P_3$ | $P_{12}$ | -21 | Remove |
| $P_5$ | $P_2$ | $P_3$ | -3 | Remove |
| $P_7$ | $P_2$ | $P_5$ | 8 | Keep |

Combined chain = $P_7 \to P_{12} \to P_1 \to P_2 \to P_5 \to P_7$

Merge two hull : Remove vertices that fall inside the combined boundary.

Final result :- $P_{10} \to P_{14} \to P_1 \to P_2 \to P_6 \to P_{10}$

Time complexity = $O(n \log n)$ with an efficient linear time merge.

Method 4 :- Graham's Scan

Selecting pivot $P_1 = (12, 1)$

Then calculate polar angle and sort them

Date:

| order | Point | Polar angle |
|---|---|---|
| 1 | ~~(0,4)~~ $P_2$ | 90° |
| 2 | ~~(-1,8)~~ $P_3$ | 97.13 |
| 3 | ~~(-3,8)~~ $P_4$ | 101.31 |
| 4 | $P_5$ | 116.57 |
| 5 | $P_7$ | 123.69 |
| 6 | $P_6$ | 124.99 |
| 7 | $P_8$ | 129.81 |
| 8 | $P_9$ | 145.01 |
| 9 | $P_{13}$ | 149.04 |
| 10 | $P_{10}$ | 151.39 |
| 11 | $P_{11}$ | 156.04 |
| 12 | $P_{12}$ | 161.57 |
| 13 | $P_{14}$ | 165.96 |
| ~~14~~ | | |

$$\theta = \tan^{-1}\left(\frac{y - y_A}{x - x_A}\right)$$

Stack table :-

| Point Process ~~ed~~ | Test |
|---|---|
| Initial | - |
| $P_4$ | cross $(P_2, P_3, P_4) = +3$ |
| $P_5$ | cross $(P_3, P_4, P_5) = -9$ |
| | cross $(P_2, P_3, P_5) = -3$ |
| | cross $(P_1, P_2, P_5) = 36$ |
| $P_7$ | cross $(P_2, P_5, P_7) = 8$ |
| $P_6$ | cross $(P_5, P_7, P_6) = -6$ cross $(P_2, P_5, P_6) = -11$ |
| | cross $(P_1, P_2, P_6) = 63$ |
| $P_8$ | cross $(P_2, P_6, P_8) = 26$ |
| $P_9$ | cross $(P_6, P_8, P_9) = -18$ |

| Action | Stack |
|---|---|
| Initialize | $P_1, P_2, P_3$ |
| push $P_4$ | $P_1, P_2, P_3, P_4$ |
| pop $P_4$ | $P_1, P_2, P_3$ |
| pop $P_3$ | $P_1, P_2$ |
| push $P_5$ | $P_1, P_2, P_5$ |
| push $P_7$ | $P_1, P_2, P_5, P_7$ |
| pop $P_7, P_5$ | $P_1, P_2$ |
| push $P_6$ | $P_1, P_2, P_6$ |
| push $P_8$ | $P_1, P_2, P_6, P_8$ |
| pop $P_8$ | $P_1, P_2, P_6$ |

Date:

| | | | |
|---|---|---|---|
| | cross $(P_2, P_6, P_9) = 24$ | push $P_9$ | $P_1, P_2, P_6, P_9$ |
| $P_{13}$ | cross $(P_6, P_9, P_{13}) = 27$ | push $P_{13}$ | $P_1, P_2, P_6, P_9, P_{13}$ |
| $P_{10}$ | cross $(P_9, P_{13}, P_{10}) = -9$ | pop $P_{13}$ | $P_1, P_2, P_6, P_9$ |
| | cross $(P_6, P_9, P_{10}) = 0$ | pop $P_9$ | $P_1, P_2, P_6$ |
| | cross $(P_2, P_6, P_{10}) = 32$ | push $P_{10}$ | $P_1, P_2, P_6, P_{10}$ |
| $P_{11}$ | cross $(P_6, P_{10}, P_{11}) = 16$ | push $P_{11}$ | $P_1, P_2, P_6, P_{10}, P_{11}$ |
| $P_{12}$ | cross $(P_{10}, P_{11}, P_{12}) = 6$ | push $P_{12}$ | $P_1, P_2, P_6, P_{10}, P_{11}, P_{12}$ |
| $P_{14}$ | cross $(P_{11}, P_{12}, P_{14}) = -9$ | pop $P_{12}$ | $P_1, P_2, P_6, P_{10}, P_{11}$ |
| | cross $(P_{10}, P_{11}, P_{14}) = -2$ | pop $P_{11}$ | $P_1, P_2, P_6, P_{10}$ |
| | cross $(P_6, P_{10}, P_{14}) = 28$ | push $P_{14}$ | $P_1, P_2, P_6, P_{10}, P_{14}$ |

Final convex hull stack = $[P_1, P_2, P_6, P_{10}, P_{14}]$

Convex Hull $\Rightarrow$ $P_1 \rightarrow P_2 \rightarrow P_6 \rightarrow P_{10} \rightarrow P_{14} \rightarrow P_1$

## Method 5: Jarvis March

$\rightarrow$ Selecting Leftmost point :- $P_{10} = (1, 7)$

$\rightarrow$ Next point on the basis of who gives the most counter clockwise turn (if cross (cur, best, c) < 0, then c is to the right of the line and becomes the new best)

From Point 10 (cross (10, cand, q)) cand = 1 {starting

| q | cross (10, cand, q) | Action |
|---|---|---|
| 2, 3, 4, 5, 6, 7, 8, 9 | 99, 82, 49, 64, 68, 42, 36, 17 (all > 0) | keep cand = 1 |
| 11 | cross (10, 1, 11) = -10 | cand = 11 |
| 12 | cross (10, 11, 12) = 6 | keep |
| 13 | cross (10, 11, 13) = 6 | keep |
| 14 | cross (10, 11, 14) = -2 | cand = 14 |

Next hull point = 14

Date:

From P14 cand = 1

| q | cross (curr, cand, q) | | action |
|---|---|---|---|
| 2, 3, 4, 5, 6 | 72, 62, 38, 56, 66 | all > 0 | keep 1 |
| 7, 8, 9, 10, 11 | 40, 38, 36, 26, 14 | | keep 1 |
| 12, 13 | 2, 14 | | keep 1 |

All +ve so cand never change Next Hull is 1

Hull = [10, 14, 1]

From P1 cand = 2

| q | cross (1, 2, q) | | Action |
|---|---|---|---|
| 3, 4, 5, 6, 7, 8 | 9, 9, 36, 63, 36, 45 | all > 0 | keep cand = 2 |
| 9, 10, 11, 12, 13, 14 | 90, 99, 81, 27, 45, 72 | | keep cand = 2 |

All +ve so cand never change Next Hull is 2

Hull = [10, 14, 1, 2]

From P2 cand = 1

| q | cross (2, cand, q) | | Action |
|---|---|---|---|
| 3 | cross (2, 1, 3) = -9 | | cand = 3 |
| 4 | cross (2, 3, 4) = 3 | | keep |
| 5 | cross (2, 3, 5) = -3 | | cand = 5 |
| 6 | cross (2, 5, 6) = -11 | | cand = 6 |
| 7, 8, 9, 10 | 25 26, 24, 32 | all > 0 | keep cand 6 |
| 11, 12, 13, 14 | 44, 59, 47, 57 | | |

Next Hull point = 6

Hull = [10, 14, 1, 2, 6

Date:

From $P_6$ cand = 1

| q | cross (6, cand, q) | action |
|---|---|---|
| 2, 3, 4, 5, 7 | 63, 46, 25, 16, 2 } all >0 | cand = 1 |
| 8 | cross (6, 1, 8) = -8 | cand = 8 |
| 9 | cross (6, 8, 9) = -18 | cand = 9 |
| 10 | cross (6, 9, 10) = 0 } collinear | tie so take cand = 10 |
| 11, 12, 13, 14 | 16, 52, 36, 28 } all >0 | keep cand = 10 |

Next Hull point = 10 } it's starting point so it's a wrap.

Result :- 10 → 14 → 1 → 2 → 6 → 10

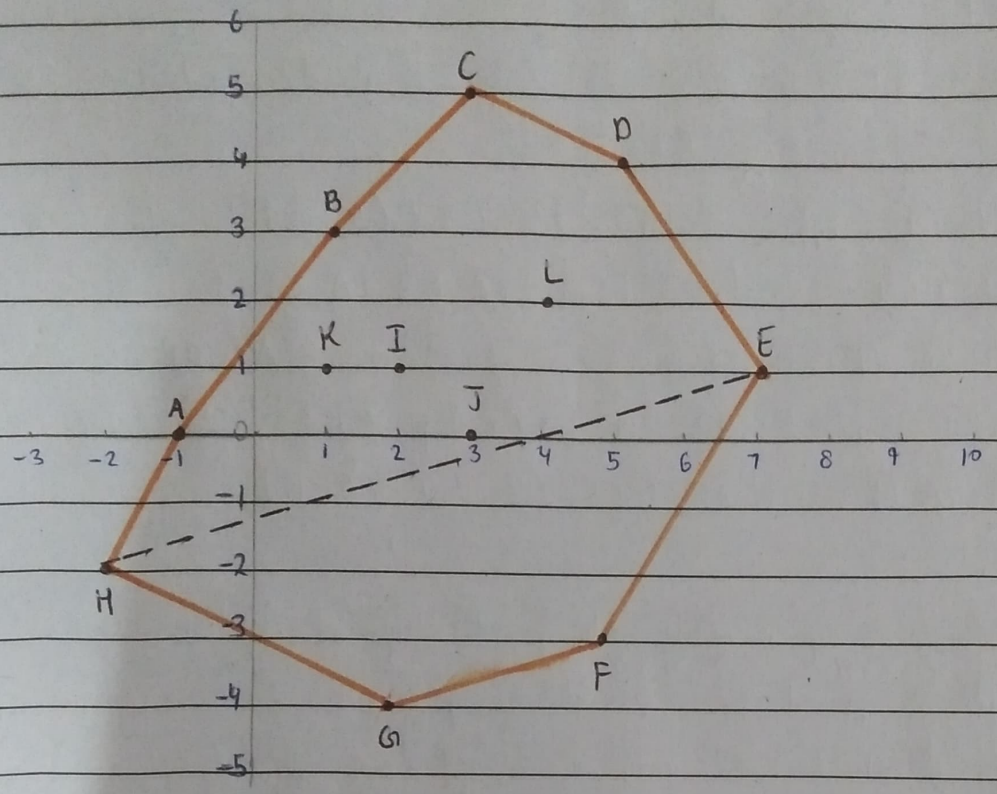
hull = 5

Final comparison

| Algorithm | Time Complexity |
|---|---|
| Brute Force | $O(n^3)$ |
| Quick Hull | Avg : $O(n \log n)$ worst $(O(n^2)$ |
| Divide & Conquer | $O(n \log n)$ |
| Graham's scan | $O(n \log n)$ |
| Jarvis March | $O(nh)$ |

Date:

Question 6




Step 1:

extreme points: leftmost H (-2, -2), rightmost = E(7, 1). No ties

Step 2: Initial partition (dotted dash line on graph)

H → E. vector (9, 3)

so cross (H, E, p) = $9(y-(-2))-3(x-(-2))$

$= 9(y+2)-3(x+2)$

$= 9y-3x+12$

| Point | A | B | C | D | E | F | G | H | I | J | K | L |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| cross | 15 | 36 | 48 | 33 | | -30 | -30 | | 15 | 3 | 18 | 18 |
| side | upper | upper | upper | upper | | lower | lower | | upper | upper | upper | upper |

Upper set: {A, B, C, D, I, J, K, L}

Lower set: {F, G}

The farthest upper point is C, with cross-product magnitude 48.

The lower points F and G tie, each with magnitude 30. we can choose either.

Date:

Applying Quick Hull recursively.

Selected C = (3, 5) relative to line HC

Direction Vector ⇒ C − H = (3+2, 5+2) = (5, 7)

cross (H, C, p) = 5(y+2) − 7(x+2) =

| Point | A | B | O | I | J | K | L |
|---|---|---|---|---|---|---|---|
| cross | 3 | 4 | −19 | −13 | −25 | −6 | −22 |
| side | outside | outside | opposite side | opposite side | opposite side | opposite side | opposite side. |

∴ B is the largest cross product 4, it's next HULL.

Selected B = (1, 3) } recursion now consider HB and BC

Direction Vector ⇒ (1+2, 3+2) = (3, 5) } for line HB

cross (H, B, P) = 3(y+2) − 5(x+2)

| Point | A |
|---|---|
| cross | 1 |
| side | outside |

- A is outside line H−B so it becomes a hull vertex.
- Now check HA
  - it has no remaining candidate on its outer side
- Now check AB
  - it has no remaining candidate on it's outer side

∴ This branch terminate and the upper hull begins:

H → A → B

For line BC

Direction Vector ⇒ C − B = (3−1, 5−3) = (2, 2)

Cross (B, C, p) = 2(y−3) − 2(x−1)

| Point | A | D | I | J | K | L |
|---|---|---|---|---|---|---|
| cross | −2 | −6 | −6 | −10 | −4 | −8 |
| side | not outside | not outside | not outside | not outside | not outside | not outside |

Date:

No point lies outside line BC, Therefore, this branch terminates.

- Find the farthest point from line CE

Direction Vector ⇒ E − C = (7−3, 1−5) = (4, −4)

cross (C, E, p) = 4(y − 5) + 4(x − 3)

| Point | D | I | J | K | L |
|---|---|---|---|---|---|
| cross | 4 | −20 | −20 | −24 | −8 |
| side | outside | not outside | not outside | not outside | not outside |

∴ D is the only candidate outside CE so D becomes next Hull vertex

- Now check CD and DE

no remaining point lies outside either segment so this branch terminates.

Upper Hull traversed from H to E ∴ is

H → A → B → C → D → E

Now Processing lower set.

from initial partition, the lower side candidates are F and G.

we have a tie so we choose G (2, −4)

Direction Vector ⇒ G − E = (2−7, −4, −1) = (−5, −5)

cross (E, G, P) = −5(y − 1) + 5(x − 7)

| Point | F |
|---|---|
| cross | 10 |
| Side | outside |

as F lies outside line EG so F becomes the next hull vertex.

- Now check the segment EF and FG

- There are no remaining candidate points outside either segment so both branches terminates.

Date:

check line GH

direction vector $\Rightarrow H - G = (-2-2, -2+4) = (-4, 2)$

cross $(G, H, F) = -4(y+4) - 2(x-2)$

| Point | F |
|---|---|
| cross | -10 |
| side | not outside |

Therefore, no additional hull vertex is found on this branch.

Lower Hull traversed is $E \rightarrow F \rightarrow G \rightarrow H$

Final convex hull :

$H \rightarrow G \rightarrow F \rightarrow E \rightarrow D \rightarrow C \rightarrow B \rightarrow A$

interior points : I, J, K, L

Verification : (every consecutive turn is left, so polygon is convex)

| | |
|---|---|
| $H \rightarrow G \rightarrow F$ | 10 |
| $G \rightarrow F \rightarrow E$ | 10 |
| $F \rightarrow E \rightarrow D$ | 14 |
| $E \rightarrow D \rightarrow C$ | 4 |

| | |
|---|---|
| $D \rightarrow C \rightarrow B$ | 6 |
| $C \rightarrow B \rightarrow A$ | 2 |
| $B \rightarrow A \rightarrow H$ | 1 |
| $A \rightarrow H \rightarrow G$ | 10 |

Question 7

i)

| Path | Endpoints |
|---|---|
| S1 | (1,1) - (5,5) |
| S2 | (1,5) - (5,1) |
| S3 | (5,5) - (5,1) |
| S4 | (2,2) - (4,4) |
| S5 | (0,0) - (0,4) |

Method : for segment $p_1p_2$ and $p_3p_4$ define cross $(a, b, c) = (b-a) \times (c-a)$ Then $d_1 = \text{cross}(p_3, p_4, p_1)$ and $d_2 = \text{cross}(p_3, p_4, p_2)$ which tell whether $p_1$ and $p_2$ are on opposite sides of line $p_3p_4$

Date:

1- (S1, S2)

p1(1,1), p2(5,5), p3(1,5), p4(5,1)

p4 - p3 = (4,-4)

p2 - p1 = (4,4)

d1 = (4,-4) • (0,-4) = -16

d2 = (4,-4) • (4,0) = 16

d3 = (4,4) • (0,4) = 16

d4 = (4,4) • (4,0) = 16

as d1.d2 < 0 and d3.d4 < 0

so this is a PROPER CROSSING

2- (S1, S3)

p1(1,1), p2(5,5), p3(5,5), p4(5,1)

p4 - p3 = (0,-4)

p2 - p1 = (4,4)

d1 = (0,-4) • (-4,-4) = -16

d2 = (0,-4) • (0,0) = 0

d3 = (4,4) • (4,4) = 0

d4 = (4,4) • (4,0) = -16

as d2 = 0 and p2 = (5,5) is an endpoint of S3. so this is ENDPOINT INTERSECTION at (5,5).

3) Solving the remaining 8 pairs like above

| Pair | P1 | P2 | P3 | P4 | cross (d1, d2, d3, d4) | |
|---|---|---|---|---|---|---|
| 3) S1, S4 | (1,1) | (5,5) | (2,2) | (4,4) | (0,0,0,0) | colinear overlap from (2,2) to (4,4) |
| 4) S1, S5 | (1,1) | (5,5) | (0,0) | (0,4) | (-4,-20,0,16) | no intersection, colinear point outside |
| 5) S2, S3 | (1,5) | (5,1) | (5,5) | (5,1) | (-16,0,16,0) | Endpoint intersection at (5,1) |
| 6) S2, S4 | (1,5) | (5,1) | (2,2) | (4,4) | (8,-8,-8,8) | Proper crossing at (3,3) |
| 7) S2, S5 | (1,5) | (5,1) | (0,0) | (0,4) | (-4,-20,-24,-8) | no intersection |
| 8) S3, S4 | (5,5) | (5,1) | (2,2) | (4,4) | (0,-8,-12,-4) | no intersection; collinear point S4 is outside |
| 9) S3, S5 | (5,5) | (5,1) | (0,0) | (0,4) | (-20,-20,-20,-20) | no intersection |
| 10) S4, S5 | (2,2) | (4,4) | (0,0) | (0,4) | (-8,-16,0,8) | no intersection, point outside S5 |

Date:

Summary

| Pair | Result | Intersection Point |
|---|---|---|
| • (S1, S2) | Proper crossing | (3, 3) |
| • (S2, S4) | Proper crossing | (3, 3) |
| • (S1, S3) | Endpoint intersection | (5, 5) |
| • (S2, S3) | Endpoint intersection | (5, 1) |
| • (S1, S4) | collinear overlap | segment (2, 2)–(4, 4) |
| • (S1, S5), (S2, S5), (S3, S4), (S3, S5), (S4, S5) | non-overlapping | none |

ii) Assumption :- every segment has $p \neq q$ (non-zero length), and arithmetic is exact integer.

Brute Force dlgorithm :-

```
ON-SEG (a, b, p):          // p is already collinear with a, b
    return min(a.x, b.x) <= p.x <= max(a.x, b.x) and
           min(a.y, b.y) <= p.y <= max(a.y, b.y)

COLLINEAR-RELATION (p1, p2, p3, p4):   // all 4 cross products are 0
    if p1.x != p2.x : use x-coordinates, else use y-coordinates   // handle vertical segm.
    lo = max(min(p1, p2), min(p3, p4))   // projection onto that coordinate
    hi = min(max(p1, p2), max(p3, p4))
    if lo > hi : return NONE          // disjoint on the same line
    if lo == hi : return ENDPOINT     // touch at exactly one point
    return OVERLAP                    // positive length overlap

SEG-RELATION (p1, p2, p3, p4):
    d1 = cross(p3, p4, p1) ; d2 = cross(p3, p4, p2)
    d3 = cross(p1, p2, p3) ; d4 = cross(p1, p2, p4)
```

Date:

```
if d1 == 0 and d2 == 0 and d3 == 0 and d4 == 0:
    return COLLINEAR-RELATION (p1, p2, p3, p4)
if sign(d1) * sign(d2) < 0 and sign(d3) * sign(d4) < 0 : return PROPER
if d1 == 0 and ON-SEG (p3, p4, p1) : return ENDPOINT
if d2 == 0 and ON-SEG ( p3, p4, p2) : return ENPPOINT
if d3 == 0 and ON-SEG ( p1, p2, p3) : return ENDPOINT
if d4 == 0 and ON-SEG (p1, p2, p4) : return ENDPOINT
return NONE
```

```
REPORT-CROSSINGS (S[1..n]):
    for i = 1 to n-1 :
        for j = i+1 to n :
            r = SEG-RELATION(S[i].p, S[i].q, S[j].p, S[j].q)
            if r != NONE :
                report (i, j, r).
```

Time Complexity:

Outer loop examines n-1 segments, and the inner loop check the remaining segment. Thus, the total numbers of pairs examined is

$$\sum_{i=1}^{n-1} (n-i) = n(n-1)/2$$

Each segment-pair test take $O(1)$ time, so the total time complexity is

$$T(n) = O(1) \cdot n(n-1)/2 = O(n^2).$$

Therefore the worst-case time complexity is $\Theta(n^2)$, since every pair must be examined regardless of whether any crossing exist.

Date:

iii) Limitations of brute force when scaling to 100,000

- with $n = 10^5$ paths, pairs $= n(n-1)/2 = 4999950000 \approx 5 \times 10^9$. 15 $10^5$ location means 50000 segments, it's about $1.25 \times 10^9$. Both are huge.
- Each test costs about 4 cross products plus comparisn so roughly $10^{10}$-$10^{11}$ elementary operations per scan. Sensor data keeps changing, so scan would have to repeat continiously. Hence too slow for real-time accident detection.
- Most pairs are far apar, so almost all this work is wasted. As brute force never uses spatial structure.

Recommended Algorithm: Plane sweep algo, Bentley-Ottomann.

- Sweep a vertical line left to right
- keep a balanced BST of the segments crossing the line, ordered by y.
- keep an event queue of segment endpoints plus intersection points discovered during the sweep.
- Two segments can only cross if they become adjacent in the BST just before the crossing, so only neighbours are tested.

Time complexity: $O((n+k)\log n)$ where $k$ is the number of reported intersections.

Justification:

- if crossing are sparse ($k = O(n)$), the cost is $O(n \log n) \approx 10^5 \times 16.6$. That's about 3000 times fewer operation than the $\approx 5 \times 10^9$ brute force tests.
- It needs $O(n)$ memory for sweep state, plus $O(k)$ only if all crossings are stored.
- If crossing are dense ($k = O(n^2)$) just to write output, so brute force is not asymptotically worse. In that extreme the sweep is $O(n^2 \log n)$ a log factor more. Hence it's called output-sensitive.

Date:

Question 8

| Algorithm | Expression | Value for $n = 10^5$ |
|---|---|---|
| Brute Force | $n^3$ | $10^{15}$ |
| Graham Scan | $n \log n = 10^5 \times 16.61$ | $\approx 1.66 \times 10^6$ |
| Jarvis ($h = 100$) | $nh = 10^5 \times 100$ | $10^7$ |

- Brute force is about $6 \times 10^8$ times more work than Graham, and Jarvis is about 6 times more than Graham

- Processed Once :- Choose Graham scan because it's $O(n \log n)$ complexity gives the lowest estimated work for the stated values Jarvis is a reasonable choice only if it's $h$ is known to be tiny.

Repeated as new points arrive :- rerunning Graham each time costs $O(n \log n)$ per arrival which is wasteful. A practical approach is to test the new point against the current hull ($O(h)$ or $O(\log h)$ with binary search). and discard if it's inside. Recompute or splice only when it falls outside.

Real Implementation Hazards.

1- Overflow :- Cross product may exceed 32 bit integer limits when coordinates are large. Use 64 bit integers or arbitrary precision arithmetic where necessary.

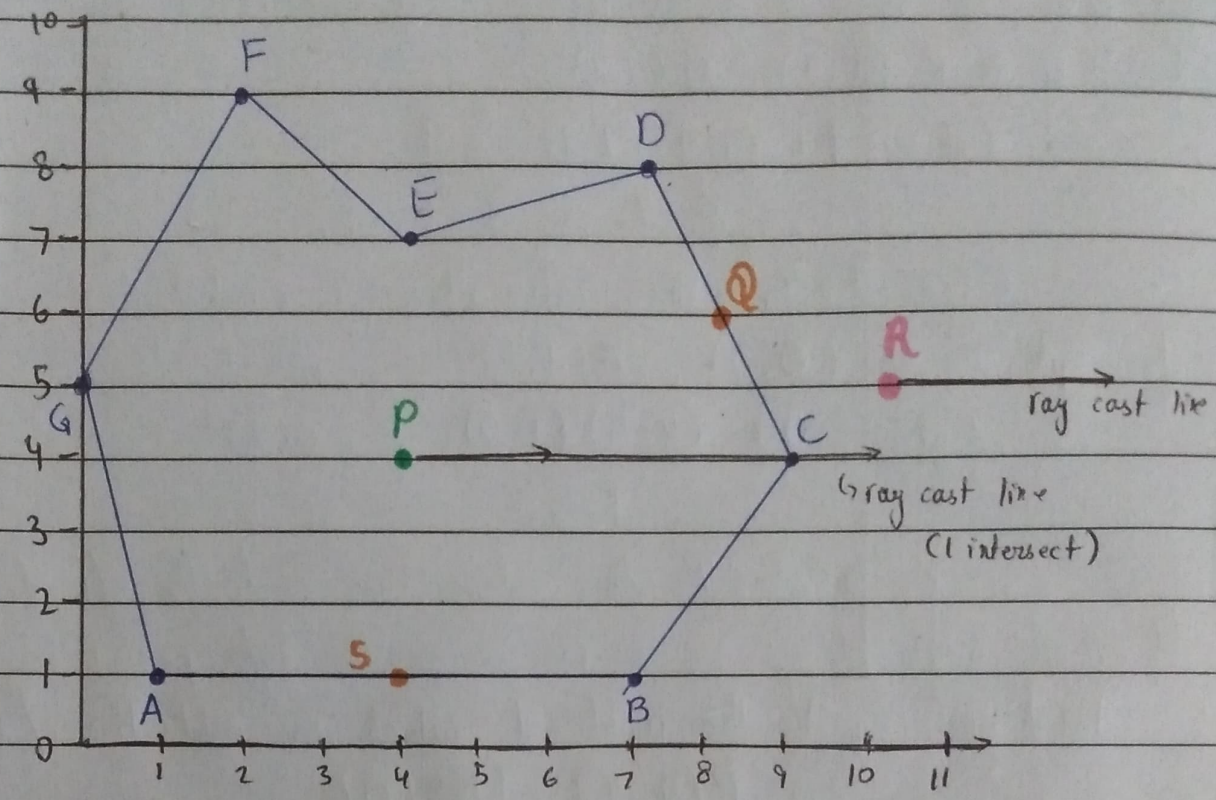
2- Duplicates :- Repeated points can cause incorrect sorting and hull construction.

~~2- Duplicates :~~ Remove duplicate points before processing.

3- Collinear Points :- Decide whether to retain or discard boundary collinear points, handle equal angle points consistently, and return a segment if all points are collinear.

Date:

Question 9




Method :- Using ray casting algorithm to classify each point as inside, outside or on the boundary of the polygon.

Approach :- For each point:

1- Check whether it lies on a polygon edge using the cross product and coordinate ranges.

2- If it's not on boundary, draw a horizontal ray to the right.

3- Count the intersections to the right of the point. An odd count means inside, and an even count means outside.

Classify P(4,4).

• Boundary check : P doesn't lie on any polygon edge so apply ray casting

Horizontal ray is at $y=4$. Examine edge BC, and CD that intersects it

LCA

~~for BC (7,1), C (4,4)~~

~~x int = 7 + (4-4)(7-7) = 7~~

~~8-4~~

~~for CD (4,4), D (7,8)~~

Date:

- for BC edge ends at $y=4$. Under half open rule, the vertex is not counted
- for CD → C (9,4) D (7,8)

$$x_{int} = 9 + \frac{(4-4)(7-9)}{8-4} = 9$$

since $9 > 4$, this intersection is counted

- for GA → G (0,5), A (1,1)

$$x_{int} = 0 + \frac{(4-5)(1-0)}{1-5} = 0.25$$

since $0.25 < 4$ this intersection is to the left of P and not counted

All other edge don't cross ray under the half open rule.

Total crossing to the right = 1 which is odd. Therefore

P(4,4) = INSIDE

Classify Q (8,6)

- Boundary check :- Check edge CD where C (9,4) and D (7,8)
  - cross product ⇒ cross (C, D, Q) = (7-9)(6-4) - (8-4)(8-9) = 0
- since cross product is zero, (C, D and Q) are collinear,
- Check whether Q lies within segment CD:

$$7 \leq 8 \leq 9 \quad \text{and} \quad 4 \leq 6 \leq 8$$

C.x  Q.x  D.y  C.y  Q.y  D.y

Both condition are satisfied. Therefore Q lies on the actual segment CD. As point lies on the polygon's edge ray casting is unnecessary.

Q (8,6) = BOUNDARY

Classify R (10,5)

- Boundary check :- R doesn't lie on any polygon edge. Apply ray casting.
- Horizontal ray is at $y = 5$. Edges are CD and FG

Date:

| edge CD → C(9,4), D(7,8) | edge FG → F(2,9), G(0,5) |
|---|---|
| $x_{int} = 9 + \frac{(5-4)(7-9)}{8-4}$ | $x_{int} = 2 + \frac{(5-9)(0-2)}{5-9}$ |
| $x_{int} = 8.5$ | $= 0$ |
| since $8.5 < 10$ the intersection is to the left of R and not counted. | since $0 < 10$, it's also to left of R and not counted. |

Total crossing to the right = 0, which is even. Therefore R(10,5) = Outside.

Classify S(4,1)

- Boundary check:- check edge AB when A(1,1) and B(7,1)
- cross-product ⇒ cross(A,B,S) = (7-1)(1-1) - (1-1)(4-1) = 0
- As cross product is zero, so (A,B,S) are collinear.
- Check whether S lies within Segment BC

$$1 \le 4 \le 7 \quad \text{and} \quad 1 \le 1 \le 1$$

A.x S.x B.x A.y S.y B.y

Both condition are satisfied. Therefore S lies on actual segment AB. As point lies on the polygons edge so ray casting is unnecessary

S(4,1) = BOUNDARY

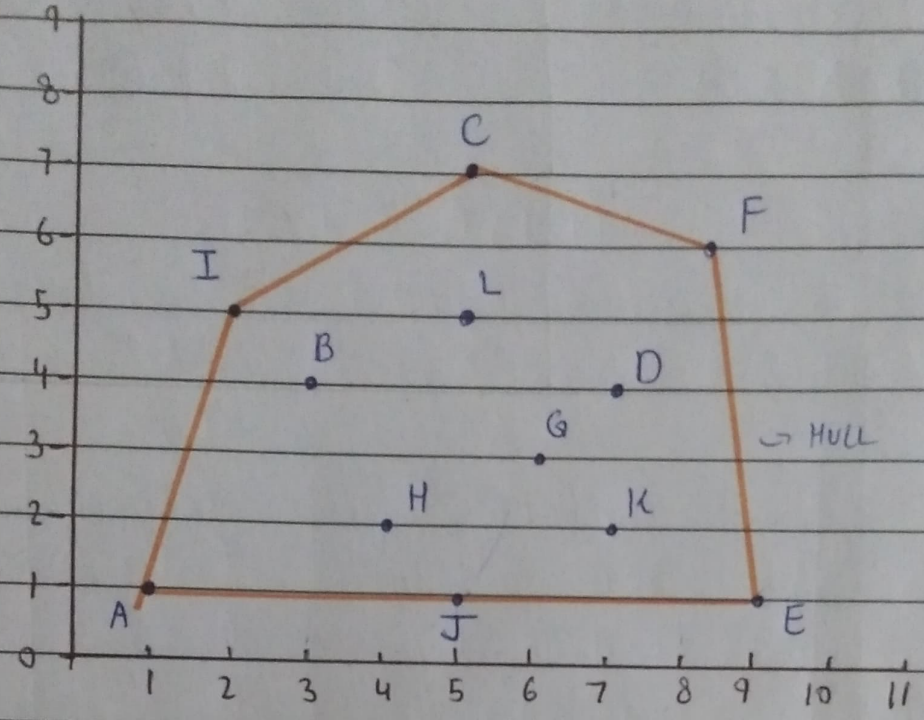
Final answer

| Attack point | Crossing to the right | Classification |
|---|---|---|
| P(4,4) | 1 (odd) | INSIDE |
| Q(8,6) | not required | Boundary |
| R(10,5) | 0 (even) | Outside |
| S(4,1) | not required | Boundary |

Date:

Question 10

i) Plot




ii) Leftmost = A (1,1) Lowest: $y=1$ at A, J and E (three way tie)

Rightmost = E (9,1) Highest = C (5,7).

iii) Brute Force + iv) orientation formula

- There are 12 point so number of pairs to check is $\binom{12}{2} = 66$. pairs
- orientation formula: $P_1(x_1, y_1)$, $P_2(x_2, y_2)$, $P_3(x_3, y_3)$

$$\text{cross}(P_1, P_2, P_3) = (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)$$

Note:- The two endpoints of the tested edge are maked with a dash.

- Candidate edge AE: A(1,1), E(9,1)

$$\text{cross}(A, E, p) = 8\,(y_p - 1)$$

orientation values are $(-, 24, 48, 24, -, 40, 16, 8, 32, 0, 8, 32)$

All values are non-negative, Point J (5,1) has orientation zero so it lies on edge AE, but it's not a necessary hull vertex

Date:

- Candidate edge ~~FC : F(8,6), C(5,7)~~ EF : E(9,1), F(8,6)

cross $(E, F, P) = (-1)(y_p - 1) - 5(x_p - 9)$

orientation values = (40, 27, 14, 7, -, -, 13, 24, 31, 20, 9, 16).

All values are positive for the remaining point. Therefore EF is a hull edge.

- Candidate edge FC: F(8,6), C(5,7)

cross $(F, C, P) = (-3)(y_p - 6) - (x_p - 8)$

orientation values: (22, 11, -, 7, 14, -, 11, 16, 9, 18, 13, 6)

All values are positive. Therefore FC is a hull edge.

- Candidate edge CI: C(5,7), I(2,5)

cross $(C, I, P) = (-3)(y_p - 7) - (-2)(x_p - 5)$

orientation values = (10, 5, -, 13, 26, 9, 14, 13, -, 18, 19, 6)

All values are positive. Therefor CI is a hull edge

- Candidate edge IA: I(2,5), A(1,1)

cross $(I, A, P) = (-1)(y_p - 5) - (-4)(x_p - 2)$

orientation values (-, 5, 10, 21, 32, 23, 18, 11, -, 16, 23, 12)

All values are positive. Therefore IA is a hull edge.

Final hull using brute force

$A \rightarrow E \rightarrow F \rightarrow C \rightarrow I \rightarrow A$

Convex Hull vertices are

A(1,1), E(9,1), F(8,6), C(5,7), I(2,5)

The point J(5,1) lies on edge AE, but it's not a necessary vertex.

The other points lie inside the hull or on it's boundary.

Date:

## v) Graham Scan

Selecting the point with smallest y-coordinate = A, E, I

Breaking tie by selecting point with smallest x-coordinate = A

Pivot → A(1,1)

Sort remaining points by polar angle

| Order | Point | Polar Angle (approx.) degree |
|---|---|---|
| 1 | J(5,1) | 0° |
| 2 | E(9,1) | 0° |
| 3 | K(7,2) | 9.46° |
| 4 | H(4,2) | 18.43 |
| 5 | G(6,3) | 21.80 |
| 6 | D(7,4) | 26.57 |
| 7 | F(8,6) | 35.54 |
| 8 | L(5,5) | 45 |
| 9 | B(3,4) | 56.31 |
| 10 | C(5,7) | 56.31 |
| 11 | I(12,5) | 75.96 |

$$\theta = \tan^{-1}\left(\frac{y - y_A}{x - x_A}\right)$$

Apply stack operation : Initialize stack push A

| Next | Test | Action | Stack |
|---|---|---|---|
| J | none | push | A, J |
| E | cross(A, J, E) = 0 → pop J push E | push | A, E |
| K | cross(A, E, K) = 8 | push | A, E, K |
| H | cross(E, K, H) = 3 | push | A, E, K, H |
| G | cross(K, H, G) = -3 → pop H; | pop | A, E, K |
| | cross(E, K, G) = -1 → pop K | pop | A, E |
| | cross(A, E, G) = 16 | push | A, E, G |
| D | cross(E, G, D) = -5 pop G | pop | A, E |

Date:

| | | | |
|---|---|---|---|
| | cross (A, E, D) = 24 | push | A, E, D |
| F | cross (E, D, F) = -7 → pop D | pop | A, E |
| | cross (A, E, F) = 40 | push | A, E, F |
| L | cross (E, F, L) = 16 | push | A, E, F, L |
| B | cross (F, L, B) = 1 | push | A, E, F, L, B |
| C | cross (L, B, C) = -4 pop B | pop | A, E, F, L |
| | cross (F, L, C) = -6 pop L | pop | A, E, F |
| | cross (E, F, C) = 14 | push | A, E, F, C |
| I | cross (F, C, I) = 9 | push | A, E, F, C, I |

HULL = A, E, F, C, I

A → E → F → C → I → A } counter clockwise order

vi) Comparing Brute Force, Graham Scan, and Quick Hull

Solving Quick Hull.

Leftmost Point = A  Rightmost point = E

Baseline initial AE.

- Find farthest point from line AE

Point

| ~~side~~Point | B | C | D | F | G | H | I | J | K | L |
|---|---|---|---|---|---|---|---|---|---|---|
| cross | 24 | 48 | 24 | 40 | 16 | 8 | 32 | 0 | 8 | 32 |

farthest point is C (5, 7). Region now divided into A to C and ~~E to~~ C to E

Date:

for edge AC

cross (A, C, I) = (5-1)(5-1) - (7-1) (2-1) = 10

point I (2, 5) lies outside the current triangle and becomes a hull vertex for B has cross 0, so it's collinear and ignored.

for edge CE

cross (C, E, F) = (9-5) (6-7) - (1-7) (8-5) = 14

point F is strictly outside so it becomes hull. While D has cross 0 and is ignored as it's collinear.

HULL :- A, E, F, C, I

Comparing all 3 algorithms give the same minimum hull. That is A, E, F, C, I. The strict sign test (>0 for QuickHull, ≤0 pops for Graham) is what keeps the collinear point J out of the vertex list. Brute force detects J as a zero cross product and must be told to drop it.

vii) Time Complexity

| Algorithm | Worst case | Note |
|---|---|---|
| Brute Force | $O(n^3)$ | C (n, 2) pairs x n-2 test |
| Graham Scan | $O(n \log n)$ | Dominated by the sort, scan is $O(n)$ |
| Quick Hull | $O(n^2)$ | Average $O(n \log n)$ |

viii) Shoelace Area

$$A = \frac{1}{2} \left| \sum_{i=1}^{n} (x_i y_{i+1} - y_i x_{i+1}) \right|$$

Date:

| Point | $x_i$ | $y_i$ | $x_i \cdot y_{i+1}$ | $y_i x_{i+1}$ |
|---|---|---|---|---|
| A | 1 | 1 | $1(1)=1$ | $1(9)=9$ |
| E | 9 | 1 | $9(6)=54$ | $1(8)=8$ |
| F | 8 | 6 | $8(7)=56$ | $6(5)=30$ |
| C | 5 | 7 | $5(5)=25$ | $7(2)=14$ |
| I | 2 | 5 | $2(1)=2$ | $5(1)=5$ |
| | Sum | | 138 | 66 |

$$\text{Shoelace} = \frac{1}{2}|138-66|$$

$$= \frac{1}{2}(72)$$

$$= 36 \text{ square units}$$ Answer